Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий и радиоэлектроники
Кафедра программной инженерии

КУРСОВАЯ РАБОТА

по дисциплине «Системы управления базами данных»

Тема «АРМ администратора гостиницы»

Пояснительная записка

_____
(оценка)

Члены комиссии

_____
(подпись) (дата)

_____
(подпись) (дата)

Руководитель
Доц. каф. ПИН
Колпаков А. А.

_____
(подпись) (дата)

Студент группы ПИНз-122
Терехов С. М.

_____
(подпись) (дата)

Муром 2026

Терехов С. М. АРМ администратора гостиницы. Курсовая работа по дисциплине «Системы управления базами данных». Муромский институт (филиал) ВлГУ, группа ПИНз-122. Муром, 2026.

Разработана настольная информационная система для администратора гостиницы Volga Grand Hotel. Программа объединяет подбор свободного номера, бронирование, регистрацию гостей, заселение, учёт услуг, оплату нескольких проживаний одним плательщиком и архив выездов. Реализация выполнена на C# с использованием .NET 8, Windows Forms и SQLite.

Для питания предусмотрен выбор конкретного дня в пределах проживания. На текущий день заказы принимаются до 10:00 для завтрака, до 14:00 для обеда и до 20:00 для ужина. Повторное питание ограничивается по гостю, виду и дате. Суммы рассчитываются в целых копейках, исторические тарифы и закрытые счета сохраняются.

Описаны модели базы версии 6, содержащей 15 таблиц, 116 атрибутов, 19 внешних ключей и 66 триггеров. В повторном запуске 88 регрессионных сценариев успешно завершились 87; отдельно рассмотрено расхождение проверки ориентации интерфейса при масштабе 125 %. Представлены реальные снимки программы, словарь данных, SQL-примеры и ссылка на исходный проект.

Ключевые слова: гостиница, бронирование, проживание, питание, SQLite, целостность данных, тарифная история, C#, Windows Forms.

Terekhov S. M. Hotel Administrator Workstation. Course project in Database Management Systems. Murom Institute, a branch of Vladimir State University, group PINz-122. Murom, 2026.

The project implements a desktop application for the Volga Grand Hotel administrator. It supports room availability search, reservations, guest registration, check-in, additional services, consolidated payment for several stays, checkout and historical records. The application uses C#, .NET 8, Windows Forms and SQLite.

Meal orders include a specific service date within the stay. Same-day orders are accepted before 10:00 for breakfast, 14:00 for lunch and 20:00 for dinner, using the computer's local time. Duplicate meals are prevented by guest, meal type and date. Monetary calculations use integer kopecks; historical rates and finalized invoice lines are preserved.

Database version 6 contains 15 application tables, 116 columns, 19 foreign keys and 66 triggers. A repeated regression run completed 87 of 88 scenarios successfully. The remaining layout-orientation assertion at 125 percent display scaling is discussed separately. The report includes actual application screenshots, data models, a data dictionary, SQL examples and a source repository link.

Keywords: hotel administration, reservation, guest stay, meal scheduling, SQLite, data integrity, historical rates, C#, Windows Forms.

Содержание

Введение 5
1 Анализ технического задания 6
1.1 Предметная область и участники процесса 6
1.2 Функциональные требования 6
1.3 Правила и ограничения предметной области 7
1.4 Выбор средств реализации 8
2 Разработка моделей данных 9
2.1 Концептуальная модель 9
2.2 Логическая модель и нормализация 10
2.3 Физическая модель и типы данных 11
2.4 Индексы и обеспечение целостности 12
3 Разработка и реализация АИС 13
3.1 Архитектура и структура проекта 13
3.2 Триггеры и алгоритмы обработки данных 13
3.3 Пользовательский интерфейс 15
3.4 Руководство пользователя 16
3.5 Ошибки и сохранность данных 18
4 Тестирование АИС 19
4.1 Среда и методика испытаний 19
4.2 Функциональные сценарии и результаты 19
4.3 Ограничения и оценка результата 21
Заключение 22
Список используемой литературы 23
Приложение А Модели и словарь данных 24
Приложение Б Примеры SQL и правил базы 35
Приложение В Исходный проект 37
Приложение Г Снимки окон программы 38

Введение

Обслуживание гостя гостиницы связано с последовательным изменением нескольких групп данных: свободного номерного фонда, бронирований, карточек проживающих, заказанных услуг и итогового расчёта. Ошибка в одной операции влияет на остальные. Например, несогласованность брони и заселения приводит к двойному размещению, а изменение текущей цены услуги без сохранения истории — к неверному счёту.

При ведении отдельных бумажных журналов или электронных таблиц администратору приходится вручную сопоставлять даты, состояние номера и расчёты. По мере накопления истории возрастает риск повторных записей, потери тарифов и несвоевременного освобождения номера. Для автоматизации необходима единая реляционная база, в которой операции связаны ключами и защищены ограничениями целостности.

Объект исследования — процесс обслуживания гостей гостиницы. Предмет исследования — организация данных и алгоритмов локального автоматизированного рабочего места администратора. Пользователем системы является администратор, который принимает обращения, регистрирует гостей и оформляет операции в настольной программе.

Цель курсовой работы — разработать АРМ администратора гостиницы, обеспечивающее согласованный учёт бронирования, проживания, услуг и окончательной оплаты. Для достижения цели необходимо проанализировать задание, определить сущности и связи, построить модели данных, реализовать интерфейс и бизнес-операции, а затем проверить основные и ошибочные сценарии.

Практический результат — локальное приложение Volga Grand Hotel для Windows на C#/.NET 8 и SQLite. Интернет-бронирование, эквайринг, фискализация и управление сетью гостиниц не реализуются.

1 Анализ технического задания

1.1 Предметная область и участники процесса

Гостиница располагает номерами категорий «Люкс», «Полулюкс», «Одноместный» и «Двухместный». Для каждого номера учитываются уникальное обозначение, этаж, вместимость, цена за сутки, изображение и эксплуатационная доступность. Стоимость устанавливается за номер целиком и не умножается на число проживающих в пределах его вместимости [1].

Администратор принимает телефонные обращения и вручную регистрирует бронь. Подключение к телефонной сети не требуется. До подтверждения размещения он проверяет даты, категорию и число мест. При заезде оформляется проживание, а состав гостей хранится отдельно от основного гостя, указанного в брони.

Гость может выступать в разных ролях: заявитель брони, основной проживающий, дополнительный проживающий, получатель питания или плательщик. Плательщик не обязан совпадать с основным гостем номера. Такое разделение необходимо для расчёта нескольких номеров одним человеком.

Во время проживания оформляются услуги, продление и переселение. При выезде программа фиксирует итог, регистрирует способ оплаты и сохраняет архив. Данные справочников могут изменяться, но завершённый расчёт должен воспроизводиться по сохранённым сведениям.

1.2 Функциональные требования

Таблица 1.1 — Требования и результат операции

Функция	**Ожидаемый результат**
Номерной фонд	Категория, вместимость, суточная цена, фото, ремонт и активность
Поиск и бронь	Свободные номера по датам, категории и числу мест; подтверждение и отмена
Гости и заезд	Карточки регистрации всех проживающих; заезд по брони или без неё

Функция	Ожидаемый результат
Изменение проживания	Продление и переселение с сохранением комнат и тарифов
Дополнительные услуги	Редактируемый каталог; сохранённые цена, название и количество
Питание	Получатель, дата внутри проживания и проверка времени заказа
Расчёт	Единая оплата одного или нескольких проживаний
История и восстановление	Архив, журнал, резервная копия и проверенное восстановление

Входные сведения вводятся через формы администратора: параметры комнаты, паспорт и контакты гостя, даты брони и выезда, состав проживающих, услуга, количество, день питания, плательщик и способ оплаты. Выходные данные представлены списками, календарём доступности, карточками регистрации и детализацией счёта.

Требование о хранении изображений выполняется полем Rooms.Photo типа BLOB. Требование об обработке на стороне базы реализовано триггерами. SQLite не предоставляет хранимых процедур серверного типа; предметные операции выполняются C#-слоем совместно с SQL-ограничениями.

1.3 Правила и ограничения предметной области

Бронь занимает полуинтервал [заезд, выезд): день заезда включается, день выезда не включается. Поэтому выезд одной брони и начало следующей могут приходиться на одну дату. Нулевые, отрицательные и некорректные периоды отклоняются. Новая и подтверждённая бронь удерживают номер; отменённая — не удерживает.

В номере допускается только одно активное проживание. Истечение планового срока не означает фактического выезда: просроченное проживание продолжает блокировать номер, включая поиск на будущие даты. При ремонте или отключении комнаты новые размещения запрещены. Вместимость проверяется по всем строкам состава проживающих.

Питание оформляется конкретному активному проживающему на один выбранный календарный день. Доступный период начинается сегодня и

заканчивается днём перед плановым выездом. Для текущего дня установлены границы: завтрак — до 10:00, обед — до 14:00, ужин — до 20:00. В момент наступления границы заказ соответствующего питания на сегодня уже отклоняется. Используется местное время компьютера.

Один гость получает не более одной порции каждого вида питания в день. Ограничение действует также при выборе другой позиции каталога с тем же видом питания и при другом проживании того же гостя. Для обычных услуг используются правила свободного повторения, одного заказа в день или одного заказа за проживание.

Итоговая оплата и выезд должны быть атомарными: нельзя сохранить платёж без согласованных распределений либо завершить проживание без итогового расчёта. Повторная оплата закрытого проживания запрещается. Для нулевого итога сохраняется закрытие и детализация без фиктивного денежного платежа.

1.4 Выбор средств реализации

Windows Forms предоставляет стандартные таблицы, формы, элементы ввода и средства печати для настольного приложения Windows [3]. Использование C# и .NET 8 соответствует характеру рабочего места администратора и существующей реализации. Интерфейс выполняется на русском языке.

SQLite хранит данные в локальном файле; Microsoft.Data.Sqlite предоставляет доступ через подключения, команды и параметры ADO.NET [4]. Для выбранного локального сценария не требуется установка отдельного сервера. Такой выбор не означает подтверждённую пригодность для интенсивной распределённой нагрузки.

Проектирование разделяет интерфейс, бизнес-операции и доступ к данным. Проверка в форме помогает пользователю, повторная проверка в бизнес-слое обеспечивает корректность команды, а ограничения SQLite защищают критичные связи и состояния при конкурентных подключениях.

2 Разработка моделей данных

2.1 Концептуальная модель

Центральным объектом предметной области является проживание. Оно объединяет основного гостя, текущий номер и необязательную исходную бронь. С ним связаны состав проживающих, история комнат и тарифов, услуги, строки окончательного счёта и архивная запись. Категория номера и каталог услуг являются самостоятельными справочниками.

Карточка гостя используется повторно в разных процессах. Отдельные связи основного гостя, получателя питания и плательщика позволяют сохранить смысл каждой операции. Например, основной гость номера может заказать обед дополнительному проживающему, а оплатить этот номер может другой зарегистрированный гость.

Сущности и их назначение перечислены в таблице 2.1. Концептуальное представление дано на рисунке А.1. Журнал аудита и настройки рассматриваются как технические сущности и не подменяют операции обслуживания.

Таблица 2.1 — Сущности реализованной системы

Сущность	**Назначение**
RoomTypes	Категории номеров
Rooms	Номера и их текущие характеристики
Guests	Карточки гостей
Bookings	Резервирование номера на период
Stays	Факты проживания
StayGuests	Состав проживающих
StaySegments	История комнат и тарифов
Services	Каталог услуг и правила повторения
ServiceOrders	Факты заказа услуг и даты питания
InvoiceLines	Зафиксированные строки закрытого счёта
Payments	Платежи и сведения о плательщике
PaymentStays	Распределение платежа по проживаниям
GuestArchive	История выездов

Сущность	Назначение
AuditLog	События изменения данных
AppSettings	Настройки приложения

2.2 Логическая модель и нормализация

Логическая модель преобразует предметные сущности в связанные отношения. Большинство таблиц имеет первичный ключ Id. Для StayGuests используется составной ключ (StayId, GuestId): один гость не может быть дважды включён в состав одного проживания. В PaymentStays первичный ключ состоит из PaymentId и StayId, а дополнительная уникальность StayId ограничивает проживание одним итоговым распределением.

Один платёж может относиться к нескольким проживаниям, но частичная оплата несколькими платежами для одного проживания не реализована. Следовательно, наличие связующей таблицы PaymentStays не означает произвольную связь «многие ко многим». При нулевом счёте распределение может отсутствовать.

Связь Stays.BookingId допускает NULL для заезда без брони. Частичный уникальный индекс запрещает два проживания по одной ненулевой ссылке на бронь. GuestArchive.StayId уникален: одному проживанию соответствует не более одной архивной записи. В ServiceOrders.GuestId ссылка необязательна в общей схеме, однако для нового питания получатель обязателен и проверяется по StayGuests.

Справочники отделены от событий, а повторяющиеся наборы гостей и тарифных периодов вынесены в отдельные отношения. Исторические снимки значений сохранены намеренно: название и цена услуги, номер комнаты в сегменте, имя плательщика и текст закрытого счёта должны оставаться прежними после редактирования справочников.

Таким образом, нормализация предотвращает хранение списков и перемещений в одном поле, а контролируемая денормализация обеспечивает

воспроизводимость истории. Полные 19 внешних ключей приведены в приложении А. Логическая модель с ключами показана на рисунке А.2.

2.3 Физическая модель и типы данных

Физическая модель соответствует SQLite версии схемы 6. В ней 15 пользовательских таблиц и 116 столбцов. Системная sqlite_sequence в это количество не включается. Типы, обязательность и ключи определены по метаданным PRAGMA table_info и foreign_key_list; словарь всех полей приведён в приложении А.

Даты операций хранятся в TEXT формата YYYY-MM-DD, отметки создания — как текстовые дата и время. Булевы признаки представлены INTEGER со значениями 0 или 1. SQLite использует классы хранения и гибкую типизацию; поэтому важные денежные значения дополнительно проверяются на целочисленный тип [5].

Новые финансовые расчёты используют целые копейки: PriceKopecks, RateKopecks, UnitPriceKopecks и AmountKopecks. Значение 90000 означает 900 рублей. Старые DailyPrice, Price и Amount типа REAL сохранены для совместимости; основанием новой арифметики служат целочисленные поля.

Изображение комнаты сохраняется в Rooms.Photo как BLOB. Изменение каталога не заменяет пользовательское фото. В таблицах ServiceOrders и StaySegments хранятся значения, необходимые для восстановления начислений, а InvoiceLines сохраняет окончательные описание, количество, тариф и сумму строки.

Переход к версии 6 изменяет правила дат питания и прикладные функции, не добавляя столбцов. До миграции создаётся резервная SQLite-копия. Существующие заказы и суммы не удаляются и не пересчитываются. Снимки физической структуры в приложении А применимы к версии 6, поскольку состав столбцов и внешних ключей сохранён.

2.4 Индексы и обеспечение целостности

Внешние ключи включаются при каждом открытии подключения [6]. Они подтверждают существование связанных записей, но не заменяют бизнес-проверки. Например, наличие GuestId и StayId само по себе не доказывает, что гость входит в состав выбранного проживания.

Уникальные ограничения защищают номер комнаты, наименование категории и услуги, нормализованный паспорт, состав проживающих и итоговое распределение оплаты. Частичные индексы ограничивают одно активное проживание комнаты, ненулевую ссылку на бронь и один открытый тарифный сегмент. В SQLite частичный индекс применяется только к строкам, удовлетворяющим условию WHERE [7].

Индексы по комнате и периоду ускоряют поиск конфликтов, а индексы ServiceOrders по гостю, виду питания и дате, а также по проживанию, услуге и дате поддерживают проверку повторений. Обычный индекс не является самостоятельным запретом дубликатов питания — этот запрет реализован триггерами.

В проверенной схеме зарегистрировано 66 триггеров. Из них 27 журналируют INSERT, UPDATE и DELETE девяти предметных таблиц; остальные 39 обеспечивают проверки и согласование состояния. Триггер может прервать недопустимую операцию через RAISE(ABORT) [8]. Для дат и нормализации используются функции, регистрируемые приложением на каждом подключении.

База не рассчитана на запись сторонним SQL-клиентом без этого контекста. При чтении схемы такие функции не требуются; при выполнении защищённых команд необходимы зарегистрированные hotel_today, hotel_now, hotel_meal_deadline, hotel_passport и hotel_meal_kind.

3 Разработка и реализация АИС

3.1 Архитектура и структура проекта

Реализация разделена на три логические части. MainForm и тематические файлы формы отвечают за вкладки, элементы ввода, выбор записей и сообщения пользователю. HotelService в каталоге Domain выполняет операции предметной области. Database в каталоге Data открывает SQLite-подключения, создаёт схему, выполняет миграции и обслуживает резервные копии.

Точка входа Program разбирает параметры запуска и инициализирует базу. Обычный запуск использует файл %LOCALAPPDATA%\HotelAdminArm\hotel.db. Параметр --database позволяет выбрать другой файл для демонстрации и испытаний. Путь к открытой базе отображается в строке состояния.

Пользовательские значения передаются в SQL через параметры. Для составных операций используется транзакция: либо все действия завершаются и фиксируются, либо изменения откатываются. Например, расчёт объединяет сохранение строк счёта, платёж, распределения, закрытие сегментов и выезд.

Таблица 3.1 — Назначение основных компонентов

Компонент	**Ответственность**
Program	Запуск, выбор базы, обработка ошибки открытия
MainForm	Интерфейс, поиск, календарь и печать
HotelService	Доступность, бронирование, проживание, услуги и расчёт
Database	Подключения, параметры, схема, миграции и копии
HotelAdminArm.Tests	Консольный набор регрессионных сценариев

3.2 Триггеры и алгоритмы обработки данных

Поиск доступности и проверка перед сохранением используют одно правило пересечения периодов: начало одного периода меньше конца другого, а конец больше начала другого. При изменении бронь исключается из сравнения с самой собой. Из результата также исключаются ремонт, отключённые комнаты и незавершённое проживание.

Заселение по брони допускается для подтверждённой записи, действующей сегодня. Основной гость, номер и согласованный тариф берутся из брони. Заезд без брони проходит проверку доступности по текущей дате. Успешная операция создаёт проживание, первый тарифный сегмент и строку основного гостя в составе проживающих; статус брони согласуется автоматически.

Продление проверяет новый срок на конфликты. Переселение закрывает прежний сегмент и создаёт новый с текущим тарифом другой комнаты. История не заменяется текущим RoomId. При расчёте день переселения не начисляется дважды, а суммарная минимальная оплата составляет одни сутки за проживание.

При добавлении услуги сохраняются её название и цена. Для питания дополнительно фиксируются получатель, вид питания и выбранный день в OrderDate. Проверка допускает сегодня и будущие дни до планового выезда. При заказе на сегодня сравнивается местное время с границей соответствующего вида питания. Проверка повторяется в триггере, поэтому ручное изменение параметров формы не обходит ограничение.

Для примера: при проживании с 10 по 14 ноября обед можно назначить на 10, 11, 12 или 13 ноября. Заказ на 10 ноября принимается только до 14:00. Второй обед тому же гостю на 11 ноября отклоняется, но обед на 12 ноября допускается. Обычная услуга независимо от состояния календаря питания сохраняется текущей датой.

Предварительный счёт суммирует тарифные сегменты и услуги. Вместе с ним вычисляется отпечаток данных. Если между просмотром и подтверждением состав счёта изменился, оплата отклоняется до повторного расчёта. Это предотвращает подтверждение устаревшей суммы.

При положительном итоге сохраняется платёж и распределение по выбранным проживаниям, затем фиксируются итоговые строки и выезд. При нулевом итоге платёж не создаётся. Триггер формирует архивную запись; полный состав гостей и история перемещений остаются в соответствующих таблицах. Повторное завершение и изменение закрытых финансовых записей блокируются.

Таблица 3.2 — Правила оформления питания

Вид	**На сегодня**	**Допустимые даты**	**Повторение**
Завтрак	До 10:00	Сегодня ≤ дата < выезд	Одна порция на гостя и день
Обед	До 14:00	Сегодня ≤ дата < выезд	Одна порция на гостя и день
Ужин	До 20:00	Сегодня ≤ дата < выезд	Одна порция на гостя и день

Время границы не включается: в 10:00:00 завтрак на сегодня уже недоступен. После границы календарь предлагает следующий допустимый день проживания. Если таких дней нет, форма сообщает о необходимости проверить или продлить проживание. При сохранении повторно проверяются текущие дата и время, поэтому оставленная открытой форма не позволяет оформить просроченный заказ.

3.3 Пользовательский интерфейс

Главное окно содержит девять вкладок: «Сводка», «Номера», «Гости», «Бронирование», «Заселение и архив», «Услуги», «Оплата», «Календарь», «Контроль и копии». Внутренние разделы отделяют список броней от поиска, заезд от активных проживаний и архива, каталог услуг от заказов, а расчёт — от истории оплат.

Таблицы поддерживают поиск и выбор записи. Редакторы используют списки связанных объектов, календарь и числовые поля. Для новой брони даты требуют явного выбора; прошедшие даты исторических броней остаются в списке, но не подставляются как даты новой операции.

В заказах питания форма содержит проживание, услугу, получателя, дату питания, количество и пояснение правила. Доступные границы календаря зависят от выбранного проживания и времени дня. Для ограниченного питания количество фиксировано равным единице. Для обычной услуги получатель питания и календарь отключаются.

В широком окне таблица и редактор располагаются рядом, в узком — один под другим. Поля прокручиваются, кнопки переносятся при недостатке места. Реальные снимки обновлённой программы представлены в приложении Г.

3.4 Руководство пользователя

Запуск. Открыть Published\HotelAdminArm.exe, сохранив рядом остальные файлы каталога. Требуется .NET Desktop Runtime 8 для Windows. Первый запуск новой базы создаёт демонстрационные записи. Для просмотра демо без изменения рабочей базы следует использовать параметр --database с отдельным именем файла.

Номера. На вкладке «Номера» выбрать запись для изменения либо начать новую. Заполнить обозначение, категорию, этаж, вместимость и цену за сутки; при необходимости добавить фотографию. Ремонт и активность задаются отдельно. Занятость определяется бронированиями и проживанием, а не произвольным ручным статусом.

Гости. Создать карточку с фамилией, именем и паспортом. Дата рождения необязательна; телефон и email проверяются при заполнении. Нормализованный паспорт не позволяет зарегистрировать дубль только за счёт пробелов или дефисов. Историю сохраняют отключением карточки вместо удаления связанных операций.

Бронирование. В разделе поиска выбрать даты заезда и выезда, категорию и минимум мест. После поиска выбрать свободный номер и перейти к созданию брони. Указать гостя и подтвердить запись. Альтернативный путь — двойной щелчок по свободному дню календаря. Отмена освобождает период; заселённая бронь защищена.

Заселение. Открыть действующую подтверждённую бронь и выполнить заезд либо оформить заезд без брони с проверкой свободного номера. Фактический заезд выполняется сегодняшней датой. Через «Проживающие» добавить остальных гостей в пределах вместимости. Кнопка «Карточка» формирует регистрационные сведения для всех проживающих.

Продление и переселение. Выбрать активное проживание и нажать соответствующую кнопку. Для продления указать более позднюю дату; для переселения выбрать другую доступную комнату. Просроченное проживание

перед переселением необходимо продлить. Прежние комнаты и тарифы сохраняются.

Питание. Во вкладке «Услуги → Заказы» выбрать проживание, завтрак, обед или ужин и фактического получателя из списка проживающих. В поле «Дата питания» указать день. Сегодня доступно до 10:00, 14:00 или 20:00 соответственно; будущая дата должна быть раньше планового выезда. Проверить правило и нажать «Добавить заказ». Дата появится в таблице и счёте.

Прочие услуги. Выбрать услугу и количество. Дата будет текущей. Для правила «Один раз в день» и «Один раз за проживание» количество равно единице; для свободно повторяемой услуги допускается от 1 до 100. Ошибочный заказ можно удалить только из открытого счёта. Изменение каталога не пересчитывает уже сохранённую цену заказа.

Оплата и выезд. На вкладке «Оплата» выделить одно или несколько активных проживаний, открыть расчёт и проверить комнаты, дни, тарифы и услуги. В этом же окне выбрать плательщика и способ оплаты, затем подтвердить. Программа сохранит итог и переместит завершённые проживания в архив. Выбор банковской карты или перевода регистрирует способ, но не выполняет банковское списание.

История и печать. Открыть детали платежа или счёт из архива. Для бумажной формы использовать «Предпросмотр / печать». В текстовом экранном просмотре текущей версии отдельные переводы строк могут отображаться слитно; печатный просмотр сохраняет раздельные строки.

Резервные копии. В «Контроль и копии» сохранить копию под новым именем. Для восстановления закрыть другие экземпляры программы и выбрать исправный файл. Перед заменой сохраняется текущая база, а выбранная копия проверяется и мигрируется отдельно. Неподходящий файл не заменяет рабочую базу.

3.5 Ошибки и сохранность данных

При отказе операции программа показывает причину: конфликт периода, недоступную комнату, превышение вместимости, повторное питание или завершение времени приёма заказов. Ошибка не считается успешным изменением. Команда проверяется повторно в момент сохранения, а не только при открытии формы.

Для обновления старой базы используется транзакционная миграция с предварительной SQLite-копией. Конфликтующие паспорта, несколько активных проживаний комнаты и повторные распределения оплаты автоматически не объединяются. Миграция откатывается и сохраняет сведения для сверки.

Вкладка контроля показывает просроченные проживания, истёкшие брони, приблизительно восстановленные старые тарифы, неоднозначные заказы питания и архив без подтверждённого расчёта. Такие исторические сведения не исправляются автоматически вымышленными платежами или выездами.

4 Тестирование АИС

4.1 Среда и методика испытаний

Проверка выполнена 10.11.2026 на Windows по текущему исходному проекту. Сборка Release завершилась без предупреждений и ошибок. Набор HotelAdminArm.Tests представляет собой консольное приложение; сценарии создают отдельные временные базы. Рабочая пользовательская база не изменяется. Проверка миграции, если исходная база доступна, выполняется на её SQLite-копии.

Для воспроизводимости большинство тестов использует моделируемую дату 08.10.2026; в проверках времени отдельно задаются часы и секунды. Дата тестовой модели не является датой выполнения курсовой работы. Испытания включают обращения к бизнес-слою, непосредственные SQL-команды, конкурентные подключения и формы Windows Forms.

Повторный полный запуск завершился результатом 87 успешных сценариев и 1 неуспешного из 88. Девять сценариев, добавленных для выбора даты питания, завершились успешно. Отдельный запуск опубликованного EXE на новой демонстрационной базе подтвердил открытие всех девяти вкладок, версию схемы 6, integrity_check = ok и отсутствие нарушений foreign_key_check.

4.2 Функциональные сценарии и результаты

Таблица 4.1 — Проверка основных операций

Сценарий	**Полученный результат**
Полный цикл обслуживания	Бронь, заезд, услуга, единая оплата, выезд и архив согласованы
Пересечение периодов	Конфликт отклонён; соседние периоды разрешены
Повторный заезд и просрочка	Второй заезд блокируется; просроченная комната не освобождается
Вместимость и ремонт	Превышение состава и новые размещения в ремонте отклонены
Продление и переселение	Конфликты проверены; тарифная история сохранена

Сценарий	Полученный результат
Сохранность расчёта	Смена справочника не меняет историческую сумму
Повторная оплата	Второй платёж закрытого проживания не создаётся
Сбой внутри транзакции	Все промежуточные изменения расчёта откатываются
Конкурентные команды	Конфликтующие брони и ограниченные заказы имеют одного победителя
Копия и миграция	Проверено сохранение данных и отказ от неподходящего файла

Таблица 4.2 — Новые сценарии даты питания

№	Проверка	Результат
1	Дата сохраняется в заказе, таблице и закрытом счёте	Успешно
2	Прошлая дата, день выезда и последующие дни запрещены	Успешно
3	Разные дни разрешены; повтор одного вида в тот же день запрещён	Успешно
4	Для всех видов проверены секунды до границы, сама граница и время после неё	Успешно
5	Обычная услуга остаётся на текущую дату после окончания питания	Успешно
6	Просроченное проживание требует продления	Успешно
7	Два одновременных будущих заказа не создают дубль	Успешно
8	Миграция версии 5 сохраняет заказы, суммы и создаёт копию	Успешно
9	Форма обновляет границы при выборе проживания и смене времени	Успешно

На отдельной демонстрационной базе, использованной для новых снимков, сохранена 91 запись в семи основных операционных таблицах: 20 номеров, 24 гостя, 10 броней, 13 проживаний, 10 услуг, 9 заказов и 5 платежей. Справочник категорий, составы проживающих, сегменты, архив и технический журнал в эту сумму не включены. Требование не менее 50 записей выполнено без подсчёта событий аудита.

Проверка прямых SQL-вставок подтверждает, что недопустимая дата и заказ после установленного времени отклоняются также на уровне SQLite. Для будущей даты сохраняется правило одной порции по гостю, виду и дню. При обновлении базы не требуется пересчитывать исторические заказы.

4.3 Ограничения и оценка результата

Неуспешный сценарий относится к проверке ориентации адаптивного интерфейса. При размере окна 1366×768 и DPI 120 (масштаб 125 %) тест ожидает вертикальный разделитель, тогда как программа выбирает горизонтальное расположение с учётом эффективной ширины. В журнале зафиксирована рабочая область 1308×508. Этот результат означает расхождение автоматического ожидания и фактического выбора компоновки; успешное прохождение данной проверки не заявляется.

Девять новых сценариев питания не зависят от этого расхождения и завершились успешно. Вместе с тем испытания не подтверждают отсутствие всех возможных дефектов и не заменяют проверку на оборудовании гостиницы. В частности, для промышленного применения необходимо отдельно согласовать системное время, режим резервирования и реальные условия эксплуатации.

В текущую версию не входят авторизация и роли, частичная оплата, авансы, возвраты, фискализация и банковский шлюз. Система предназначена для локального учёта. Известное отображение переводов строк в экранном текстовом просмотре не изменяет сохранённую детализацию и печатные формы.

Основные предметные требования подтверждены: подбор по параметрам, цена за номер, регистрация состава гостей, редактируемые услуги, единая оплата нескольких проживаний, BLOB-изображения, триггеры и архив. Результаты позволяют использовать программу как учебную реализацию АРМ с явно указанными границами.

Заключение

В ходе курсовой работы разработано АРМ администратора гостиницы Volga Grand Hotel. Приложение объединяет поиск свободного номера, бронирование, заселение, состав проживающих, учёт услуг, окончательную оплату и архив. Разделение справочников и операций позволяет сохранять историю обслуживания после изменения текущих данных.

Построены концептуальная, логическая и физическая модели. Реализованная схема версии 6 содержит 15 пользовательских таблиц, 116 атрибутов, 19 внешних ключей и 66 триггеров. Ключи, индексы и транзакции защищают связи и критичные состояния. Денежные расчёты выполняются в целых копейках; тарифные сегменты и итоговые строки обеспечивают воспроизводимость счёта.

Дополнительно реализован выбор дня питания в пределах проживания. Для сегодняшней даты действуют границы 10:00, 14:00 и 20:00. Ограничения проверяются интерфейсом, бизнес-слоем и SQLite, а повторный заказ контролируется по гостю, виду питания и календарному дню.

Повторный запуск регрессий дал 87 успешных результатов из 88. Все девять новых сценариев питания пройдены. Отдельно зафиксировано расхождение проверки ориентации при масштабе 125 %, которое требует дальнейшего уточнения. Целостность демонстрационной базы подтверждена; объём предметных данных превышает требуемые 50 записей.

Результат соответствует основным функциональным требованиям задания в пределах локальной версии. Дальнейшее развитие связано с уточнением проверки адаптивности, улучшением экранного текстового просмотра, разграничением доступа и расширением расчётных сценариев. Исходный проект предоставляется по ссылке в приложении В.

Список используемой литературы

1. Задание на курсовую работу «АРМ администратора гостиницы». Студент Терехов С. М., группа ПИНз-122. МИ ВлГУ. Материалы индивидуального задания.

2. HotelAdminArm. Исходный проект, схема базы версии 6 и регрессионный набор HotelAdminArm.Tests. Локальный комплект курсовой работы. Проверено 10.11.2026.

3. Microsoft. Windows Forms Overview [Электронный ресурс]. URL: https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/ (дата обращения: 10.11.2026).

4. Microsoft. Microsoft.Data.Sqlite overview [Электронный ресурс]. URL: https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/ (дата обращения: 10.11.2026).

5. SQLite. Datatypes In SQLite [Электронный ресурс]. URL: https://www.sqlite.org/datatype3.html (дата обращения: 10.11.2026).

6. SQLite. Foreign Key Support [Электронный ресурс]. URL: https://www.sqlite.org/foreignkeys.html (дата обращения: 10.11.2026).

7. SQLite. Partial Indexes [Электронный ресурс]. URL: https://www.sqlite.org/partialindex.html (дата обращения: 10.11.2026).

8. SQLite. CREATE TRIGGER [Электронный ресурс]. URL: https://www.sqlite.org/lang_createtrigger.html (дата обращения: 10.11.2026).

9. Hotel Administrator Workstation. Репозиторий проекта [Электронный ресурс]. URL: https://github.com/ignil722/hotel-admin-arm.git.

Приложение А Модели и словарь данных

Концептуальная и логическая модели представлены обзором сущностей и схемой ключей. Физические фрагменты показывают фактические имена и типы столбцов. Версия 6 меняет правила питания без изменения состава таблиц, поэтому приведённая структура согласуется с метаданными текущей базы.

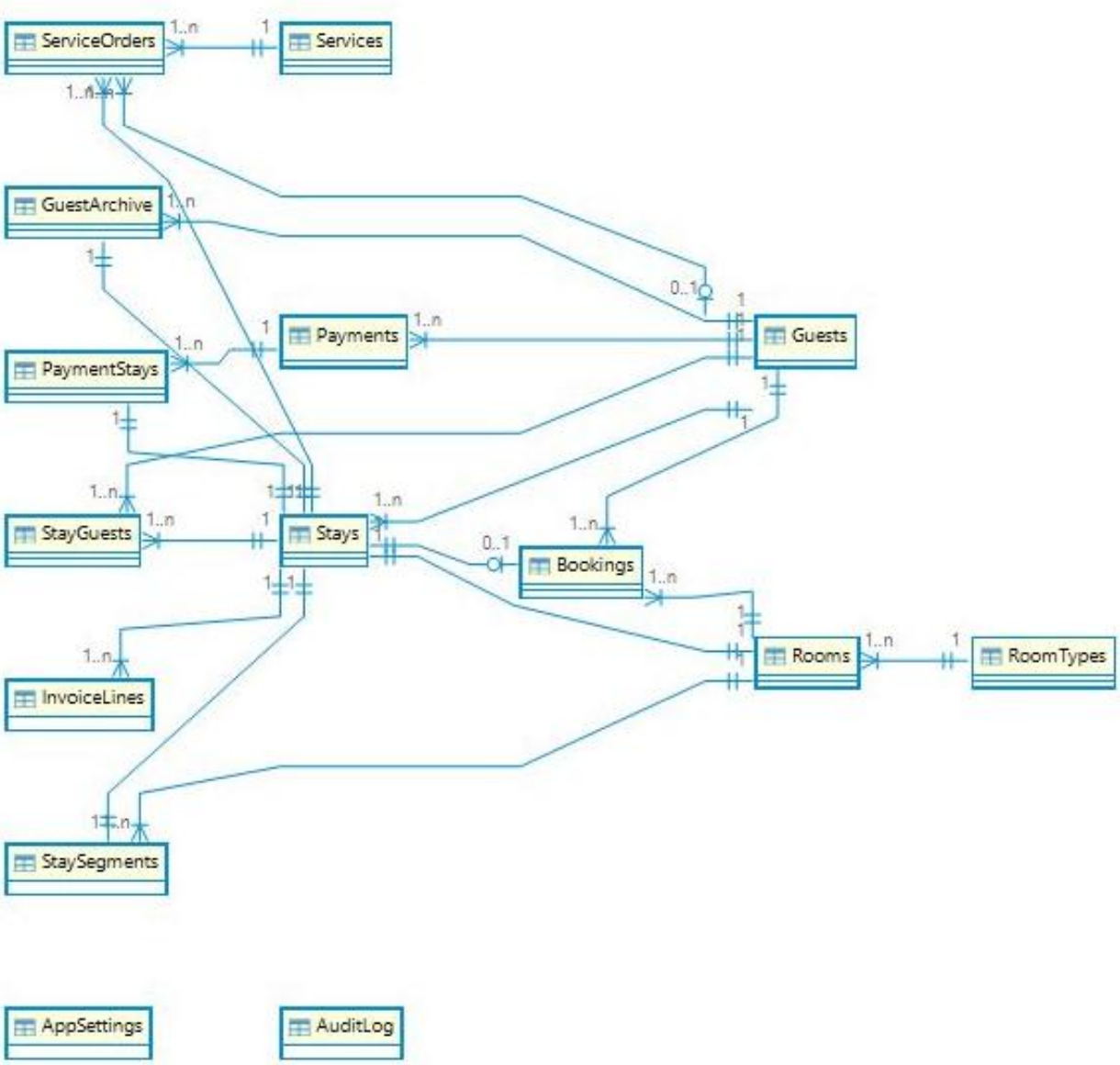


Рисунок А.1 — Концептуальная структура сущностей

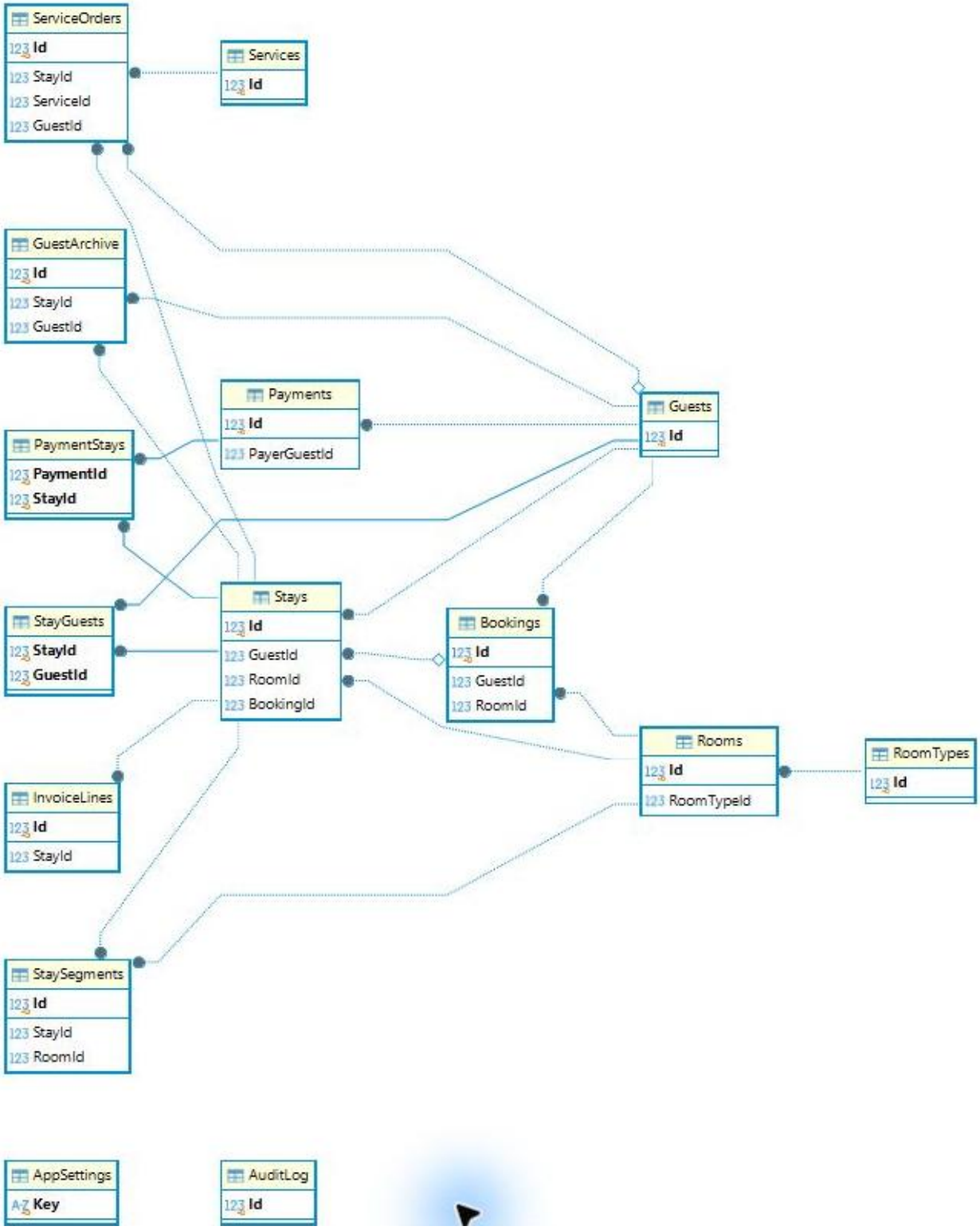


Рисунок А.2 — Логическая модель и внешние ключи

В схеме объявлено 19 внешних ключей. Значок ключа выделяет первичный ключ. Точная обязательность и действия удаления приведены в таблице А.16; существование ссылки не требует наличия дочерних строк у каждого родителя. У AppSettings и AuditLog внешних ключей нет.

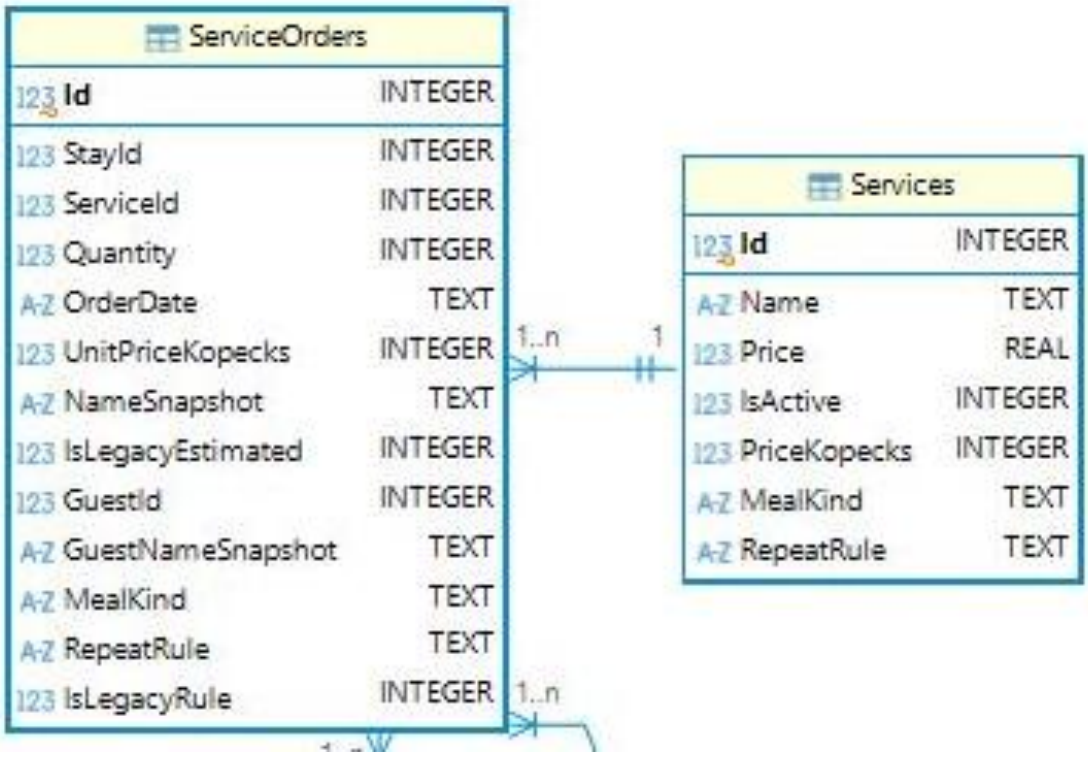


Рисунок А.3 — Каталог и заказы услуг

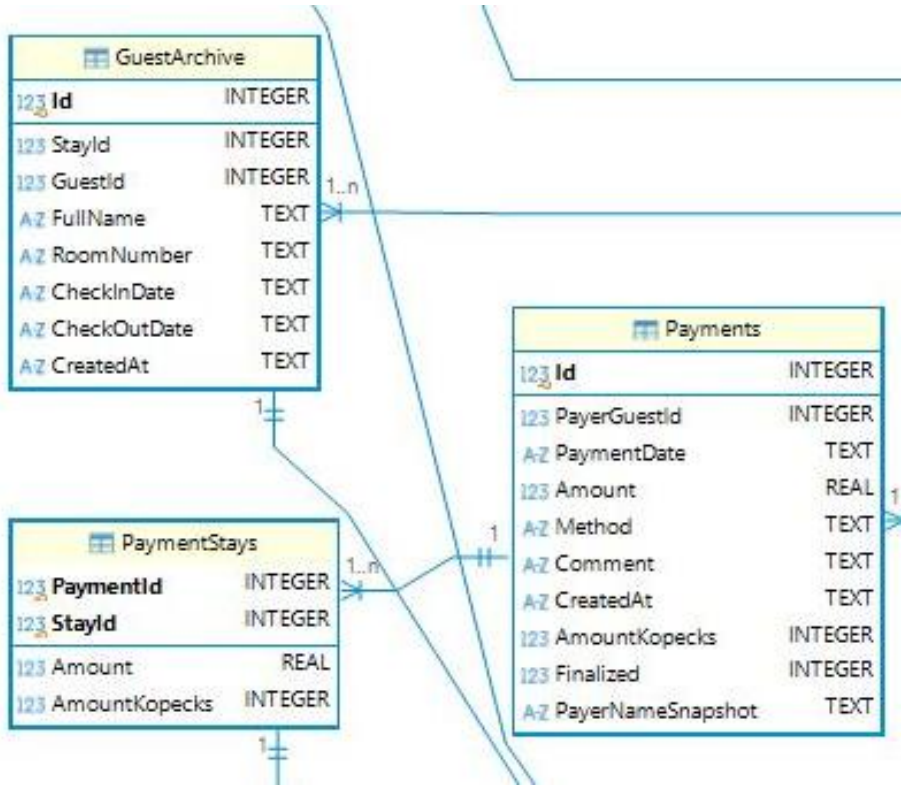


Рисунок А.4 — Платежи и архив



Рисунок А.5 — Карточки гостей

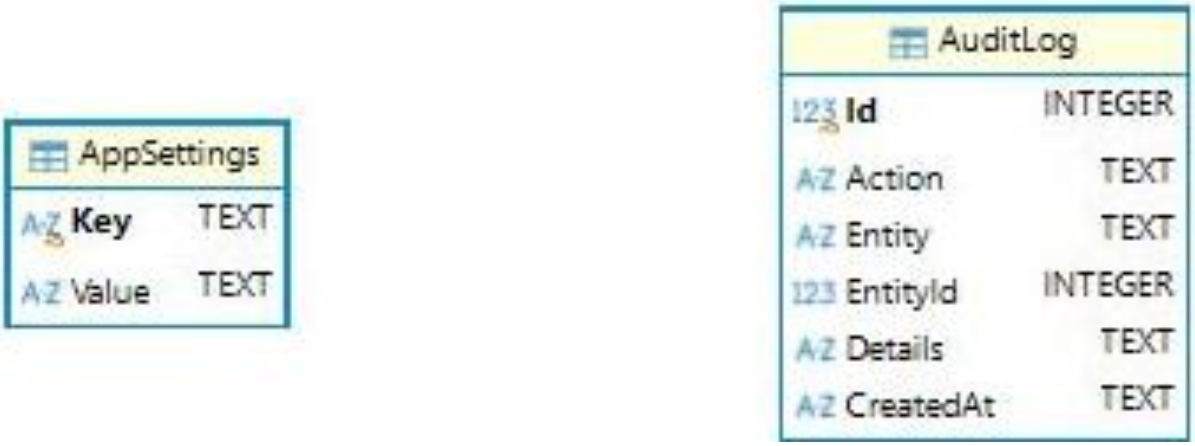


Рисунок А.6 — Настройки и журнал

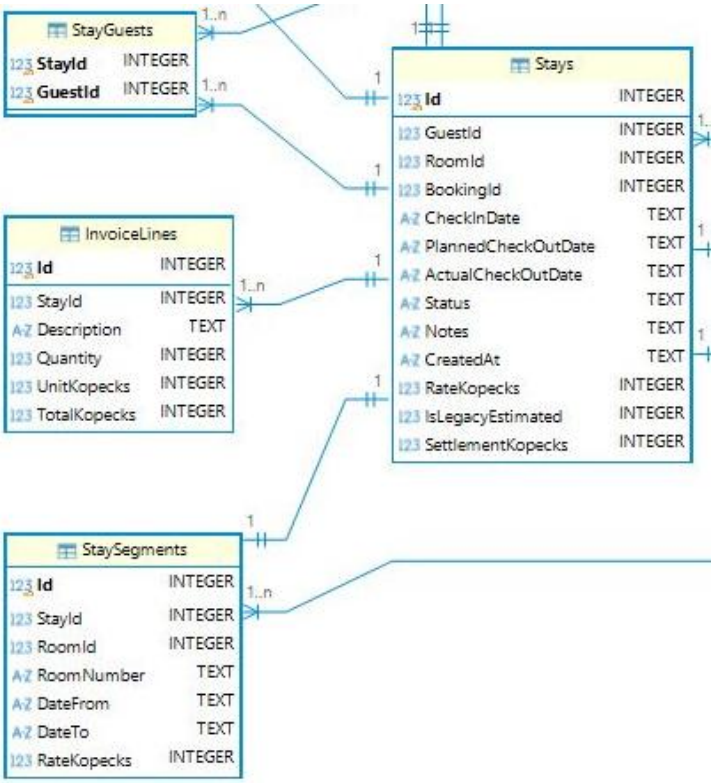


Рисунок А.7 — Проживания и начисления

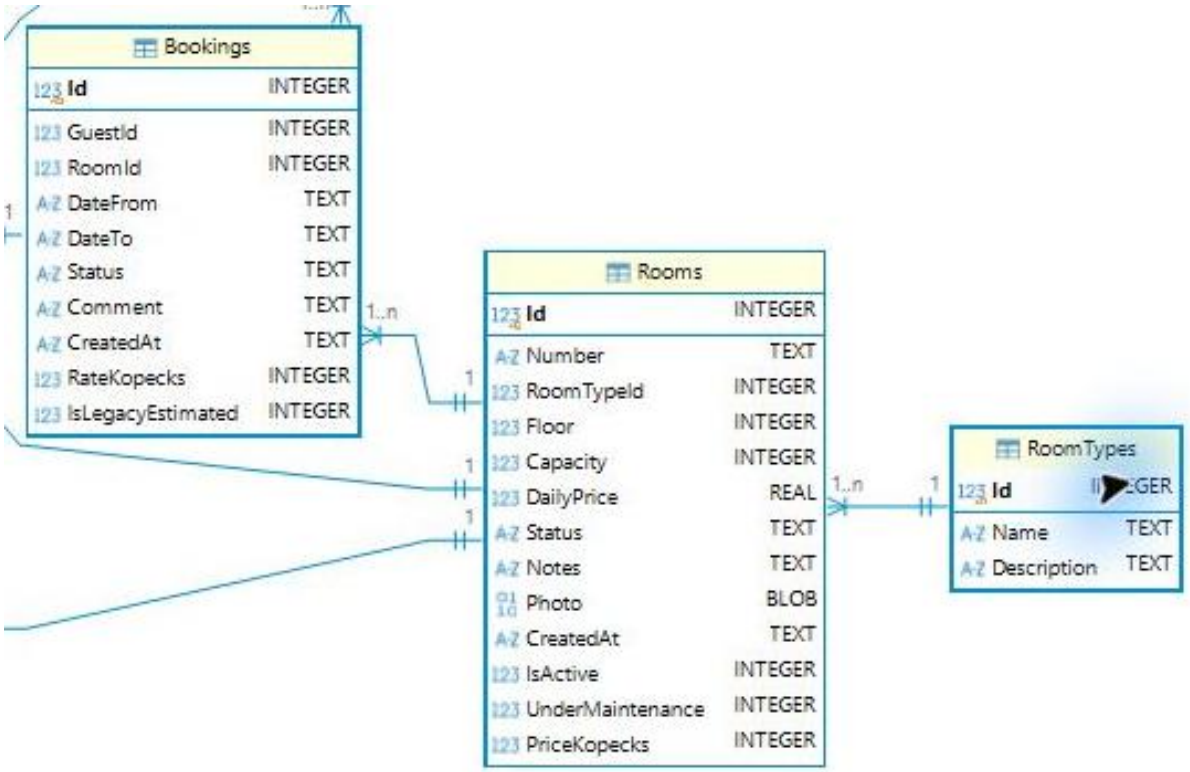


Рисунок А.8 — Брони и номера

Словарь атрибутов

Ниже приведены все 116 столбцов 15 пользовательских таблиц. PK — первичный ключ; FK — внешний ключ; NN — обязательное значение; U — дополнительная уникальность. Для составного PK указана позиция. Ограничения отдельных состояний и правила триггеров описаны в основном тексте.

Таблица А.1 — RoomTypes

Поле	**Тип и ключи**	**Назначение**
Id	INTEGER PK, NN	Идентификатор записи
Name	TEXT NN	Наименование
Description	TEXT NN	Описание

Таблица А.2 — Rooms

Поле	**Тип и ключи**	**Назначение**
Id	INTEGER PK, NN	Идентификатор записи
Number	TEXT NN	Номер комнаты
RoomTypeId	INTEGER FK, NN	Категория номера
Floor	INTEGER NN	Этаж
Capacity	INTEGER NN	Вместимость
DailyPrice	REAL NN	Совместимая цена, руб.
Status	TEXT NN	Состояние записи
Notes	TEXT	Примечание
Photo	BLOB	Изображение номера
CreatedAt	TEXT NN	Момент создания
IsActive	INTEGER NN	Активность 0 или 1
UnderMaintenance	INTEGER NN	Признак ремонта 0 или 1
PriceKopecks	INTEGER NN	Действующая цена, коп.

Таблица А.3 — Guests

Поле	**Тип и ключи**	**Назначение**
Id	INTEGER PK, NN	Идентификатор записи
LastName	TEXT NN	Фамилия
FirstName	TEXT NN	Имя
MiddleName	TEXT	Отчество
BirthDate	TEXT	Дата рождения

Поле	Тип и ключи	Назначение
Passport	TEXT NN	Паспорт как введён
Phone	TEXT	Телефон
Email	TEXT	Электронная почта
Address	TEXT	Адрес
CreatedAt	TEXT NN	Момент создания
IsActive	INTEGER NN	Активность 0 или 1
PassportKey	TEXT NN	Нормализованный паспорт

Таблица А.4 — Bookings

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
GuestId	INTEGER FK, NN	Гость
RoomId	INTEGER FK, NN	Номер
DateFrom	TEXT NN	Начало периода
DateTo	TEXT NN	Конец периода
Status	TEXT NN	Состояние записи
Comment	TEXT	Комментарий
CreatedAt	TEXT NN	Момент создания
RateKopecks	INTEGER NN	Сохранённый тариф, коп.
IsLegacyEstimated	INTEGER NN	Признак приблизительной истории

Таблица А.5 — Stays

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
GuestId	INTEGER FK, NN	Основной гость
RoomId	INTEGER FK, NN	Номер
BookingId	INTEGER FK	Исходная бронь
CheckInDate	TEXT NN	Дата заезда
PlannedCheckOutDate	TEXT NN	Плановый выезд
ActualCheckOutDate	TEXT	Фактический выезд
Status	TEXT NN	Состояние записи
Notes	TEXT	Примечание
CreatedAt	TEXT NN	Момент создания
RateKopecks	INTEGER NN	Сохранённый тариф, коп.
IsLegacyEstimated	INTEGER NN	Признак приблизительной истории
SettlementKopecks	INTEGER	Сохранённый итог выезда, коп.

Таблица А.6 — StayGuests

Поле	Тип и ключи	Назначение
StayId	INTEGER PK1, FK, NN	Проживание
GuestId	INTEGER PK2, FK, NN	Гость

Таблица А.7 — StaySegments

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
StayId	INTEGER FK, NN	Проживание
RoomId	INTEGER FK, NN	Номер
RoomNumber	TEXT NN	Исторический номер комнаты
DateFrom	TEXT NN	Начало периода
DateTo	TEXT	Конец сегмента; NULL — открыт
RateKopecks	INTEGER NN	Сохранённый тариф, коп.

Таблица А.8 — Services

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
Name	TEXT NN	Наименование
Price	REAL NN	Совместимая цена, руб.
IsActive	INTEGER NN	Активность 0 или 1
PriceKopecks	INTEGER NN	Действующая цена, коп.
MealKind	TEXT NN	Вид питания: None, Breakfast, Lunch или Dinner
RepeatRule	TEXT NN	Повторение: Repeatable, Daily или Once

Таблица А.9 — ServiceOrders

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
StayId	INTEGER FK, NN	Проживание
ServiceId	INTEGER FK, NN	Услуга каталога
Quantity	INTEGER NN	Количество
OrderDate	TEXT NN	Дата предоставления: для питания выбирается в пределах проживания; для обычной услуги — текущая дата
UnitPriceKopecks	INTEGER NN	Цена заказа, коп.
NameSnapshot	TEXT NN	Историческое название услуги

Поле	Тип и ключи	Назначение
IsLegacyEstimated	INTEGER NN	Признак приблизительной истории
GuestId	INTEGER FK	Получатель питания
GuestNameSnapshot	TEXT NN	Историческое ФИО получателя
MealKind	TEXT NN	None / Breakfast / Lunch / Dinner
RepeatRule	TEXT NN	Repeatable / Daily / Once
IsLegacyRule	INTEGER NN	Признак старого заказа

Таблица А.10 — InvoiceLines

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
StayId	INTEGER FK, NN	Проживание
Description	TEXT NN	Описание
Quantity	INTEGER NN	Количество
UnitKopecks	INTEGER NN	Цена единицы строки, коп.
TotalKopecks	INTEGER NN	Итог строки, коп.

Таблица А.11 — Payments

Поле	Тип и ключи	Назначение
Id	INTEGER PK, NN	Идентификатор записи
PayerGuestId	INTEGER FK, NN	Гость-плательщик
PaymentDate	TEXT NN	Дата оплаты
Amount	REAL NN	Совместимая сумма, руб.
Method	TEXT NN	Способ оплаты
Comment	TEXT	Комментарий
CreatedAt	TEXT NN	Момент создания
AmountKopecks	INTEGER NN	Сумма, коп.
Finalized	INTEGER NN	Финализация платежа 0 или 1
PayerNameSnapshot	TEXT NN	Историческое ФИО плательщика

Таблица А.12 — PaymentStays

Поле	Тип и ключи	Назначение
PaymentId	INTEGER PK1, FK, NN	Платёж
StayId	INTEGER PK2, FK, NN	Проживание
Amount	REAL NN	Совместимая сумма, руб.
AmountKopecks	INTEGER NN	Сумма, коп.

Таблица А.13 — GuestArchive

Поле	**Тип и ключи**	**Назначение**
Id	INTEGER PK, NN	Идентификатор записи
StayId	INTEGER FK, NN	Проживание
GuestId	INTEGER FK, NN	Гость
FullName	TEXT NN	Историческое ФИО основного гостя
RoomNumber	TEXT NN	Исторический номер комнаты
CheckInDate	TEXT NN	Дата заезда
CheckOutDate	TEXT NN	Дата выезда
CreatedAt	TEXT NN	Момент создания

Таблица А.14 — AuditLog

Поле	**Тип и ключи**	**Назначение**
Id	INTEGER PK, NN	Идентификатор записи
Action	TEXT NN	Действие журнала
Entity	TEXT NN	Имя таблицы объекта
EntityId	INTEGER	Код объекта без FK
Details	TEXT	Описание изменения
CreatedAt	TEXT NN	Момент создания

Таблица А.15 — AppSettings

Поле	**Тип и ключи**	**Назначение**
Key	TEXT PK	Ключ настройки
Value	TEXT NN	Значение настройки

AppSettings.Key — текстовый PK без явно заданного NOT NULL; приложение использует непустые ключи.

Таблица А.16 — Внешние ключи реализованной схемы

Дочернее поле	Родитель	NULL	ON DELETE
Rooms.RoomTypeId	RoomTypes.Id	Нет	NO ACTION
Bookings.RoomId	Rooms.Id	Нет	RESTRICT
Bookings.GuestId	Guests.Id	Нет	RESTRICT
Stays.BookingId	Bookings.Id	Да	SET NULL
Stays.RoomId	Rooms.Id	Нет	RESTRICT
Stays.GuestId	Guests.Id	Нет	RESTRICT
StayGuests.GuestId	Guests.Id	Нет	RESTRICT
StayGuests.StayId	Stays.Id	Нет	RESTRICT
StaySegments.RoomId	Rooms.Id	Нет	RESTRICT
StaySegments.StayId	Stays.Id	Нет	RESTRICT
ServiceOrders.ServiceId	Services.Id	Нет	RESTRICT
ServiceOrders.StayId	Stays.Id	Нет	CASCADE
ServiceOrders.GuestId	Guests.Id	Да	RESTRICT
InvoiceLines.StayId	Stays.Id	Нет	RESTRICT
Payments.PayerGuestId	Guests.Id	Нет	RESTRICT
PaymentStays.StayId	Stays.Id	Нет	RESTRICT
PaymentStays.PaymentId	Payments.Id	Нет	CASCADE
GuestArchive.GuestId	Guests.Id	Нет	RESTRICT
GuestArchive.StayId	Stays.Id	Нет	RESTRICT

Защитные триггеры могут отклонить удаление раньше, чем будет применено действие внешнего ключа. Уникальность BookingId, StayId и активного RoomId дополнительно ограничивается индексами.

Таблица А.17 — Явные индексы

Индекс	Таблица и условие
UX_Stays_BookingId	Stays(BookingId) WHERE BookingId IS NOT NULL
UX_Guests_PassportKey	Guests(PassportKey)
UX_PaymentStays_Stay	PaymentStays(StayId)
UX_StaySegments_Open	StaySegments(StayId) WHERE DateTo IS NULL
IX_Bookings_RoomPeriod	Bookings(RoomId,Status,DateFrom,DateTo)
IX_Stays_RoomStatus	Stays(RoomId,Status)
IX_Orders_Stay	ServiceOrders(StayId)
UX_Stays_ActiveRoom	Stays(RoomId) WHERE Status='Проживает'
IX_Orders_MealGuestDate	ServiceOrders(GuestId,MealKind,OrderDate)
IX_Orders_ServiceStayDate	ServiceOrders(StayId,ServiceId,OrderDate)

Приложение Б Примеры SQL и правил базы

Примеры отражают запросы и ограничения фактической реализации. Значения пользователя передаются параметрами. При работе с защищёнными командами подключение должно регистрировать прикладные функции программы.

Б.1 Список номеров с категорией и ценой

```
SELECT r.Number, rt.Name AS RoomType, r.Capacity,
       r.PriceKopecks / 100.0 AS DailyRubles
FROM Rooms r JOIN RoomTypes rt ON rt.Id = r.RoomTypeId
WHERE r.IsActive = 1 AND r.UnderMaintenance = 0
ORDER BY r.Number;
```

Запрос выводит эксплуатационно доступные номера. Он не заменяет поиск свободного периода: занятость по броням и проживаниям проверяется отдельно.

Б.2 Поиск доступного номера на период

```
SELECT r.Id, r.Number
FROM Rooms r
WHERE r.IsActive = 1 AND r.UnderMaintenance = 0
  AND r.Capacity >= @capacity
  AND (@type = 0 OR r.RoomTypeId = @type)
  AND NOT EXISTS (
    SELECT 1 FROM Bookings b
    WHERE b.RoomId = r.Id
      AND b.Status IN ('Новое', 'Подтверждено')
      AND @from < b.DateTo AND @to > b.DateFrom)
  AND NOT EXISTS (
    SELECT 1 FROM Stays s
    WHERE s.RoomId = r.Id AND s.Status = 'Проживает'
      AND @to > s.CheckInDate
      AND (s.PlannedCheckOutDate <= hotel_today()
           OR @from < s.PlannedCheckOutDate));
```

Параметры @from и @to задают полуинтервал. При редактировании прикладной запрос дополнительно исключает текущую бронь или проживание из сравнения с самим собой.

Б.3 Проверка дня и времени питания в триггере

```
SELECT CASE WHEN NEW.MealKind <> 'None'
  AND (NEW.OrderDate < hotel_today()
       OR NEW.OrderDate >= (
         SELECT PlannedCheckOutDate
         FROM Stays WHERE Id = NEW.StayId))
THEN RAISE(ABORT, 'Дата питания вне доступного периода') END;

SELECT CASE WHEN NEW.MealKind <> 'None'
  AND NEW.OrderDate = hotel_today()
  AND time(hotel_now()) >= hotel_meal_deadline(NEW.MealKind)
THEN RAISE(ABORT, 'Приём заказов на сегодня завершён') END;
```

Фрагмент показывает условия существующего order_rules_insert с сокращёнными сообщениями. hotel_meal_deadline возвращает 10:00:00, 14:00:00 или 20:00:00. Проверки активности, получателя, цены, количества и повторения выполняются в том же наборе правил.

Б.4 Поиск повторного питания

```
SELECT COUNT(*)
FROM ServiceOrders
WHERE MealKind = @meal AND OrderDate = @date
  AND (GuestId = @guest
       OR (GuestId IS NULL AND StayId = @stay));
```

Ненулевой результат запрещает новый заказ. Условие с NULL сохраняет защиту от неоднозначных старых заказов, у которых получатель не был установлен.

Б.5 Проверка целостности и версии

```
PRAGMA user_version;
PRAGMA integrity_check;
PRAGMA foreign_key_check;
```

Для проверенной базы получены соответственно 6, ok и пустой набор нарушений внешних ключей.

Приложение В Исходный проект

Исходные тексты проекта предоставляются через репозиторий GitHub [9]. В записке приведена ссылка на проект вместо полного листинга программы.

https://github.com/ignil722/hotel-admin-arm.git

Основное решение — Source/HotelAdminArm.sln. Исполняемое приложение находится в проекте HotelAdminArm, регрессионные сценарии — в HotelAdminArm.Tests. Каталоги Data и Domain содержат схему, миграции и предметные операции; тематические части MainForm — формы администратора.

Для сборки исходников требуется .NET SDK с поддержкой целевой платформы net8.0-windows и восстановление пакетов NuGet. Команды выполняются из корня проекта.

```
dotnet build Source/HotelAdminArm.sln -c Release
dotnet run --project Source/HotelAdminArm.Tests -c Release
dotnet publish Source/HotelAdminArm/HotelAdminArm.csproj   -c Release -o Published
```

Выбор файла базы не требует изменения исходного кода:

```
Published\HotelAdminArm.exe --database C:\Temp\hotel-demo.db
```

Для создания отдельной демонстрационной базы следует указать новое имя файла. Удалять рабочую hotel.db для повторного заполнения демо не требуется.

Приложение Г Снимки окон программы

Снимки получены из обновлённой программы на отдельной демонстрационной базе.

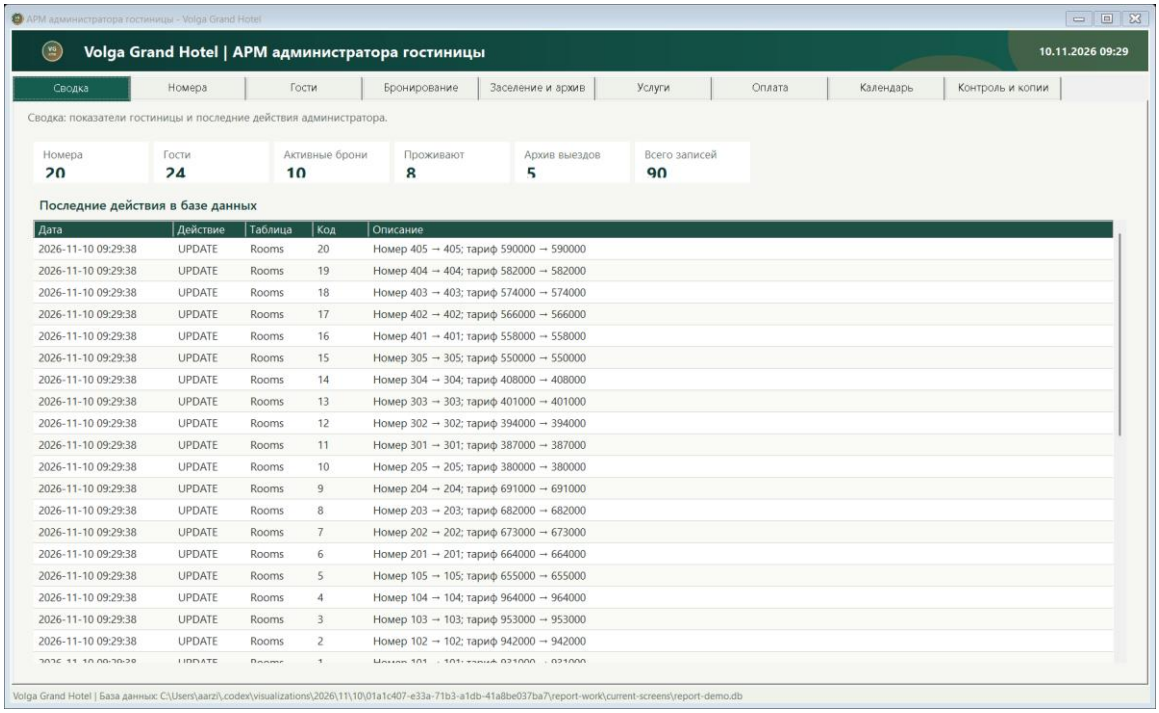


Рисунок Г.1 — Главная сводка

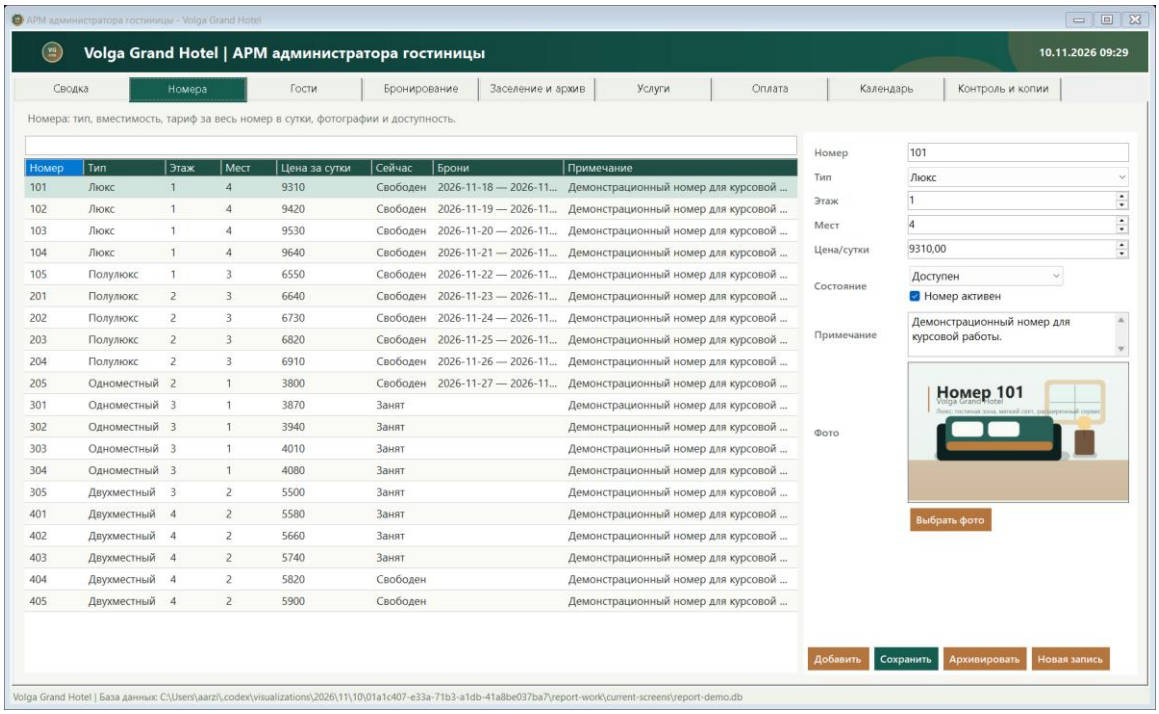


Рисунок Г.2 — Номерной фонд и карточка номера

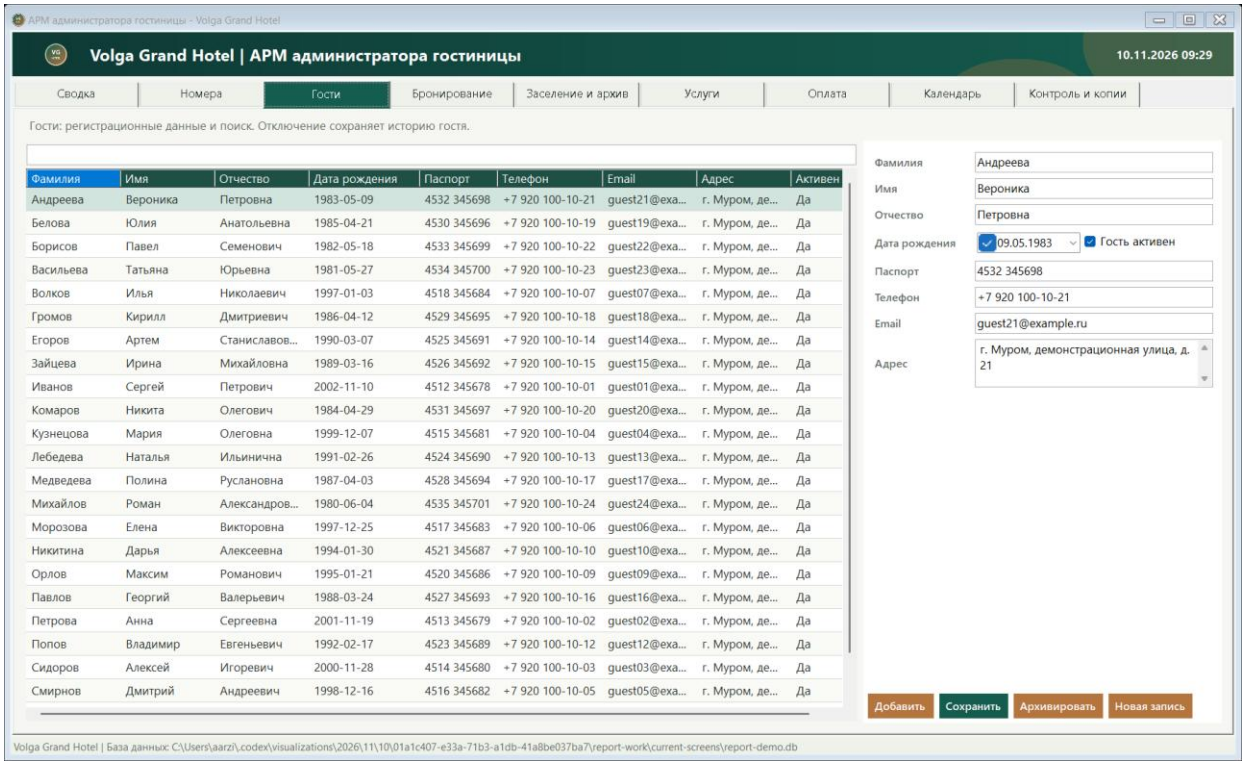


Рисунок Г.3 — Карточки гостей

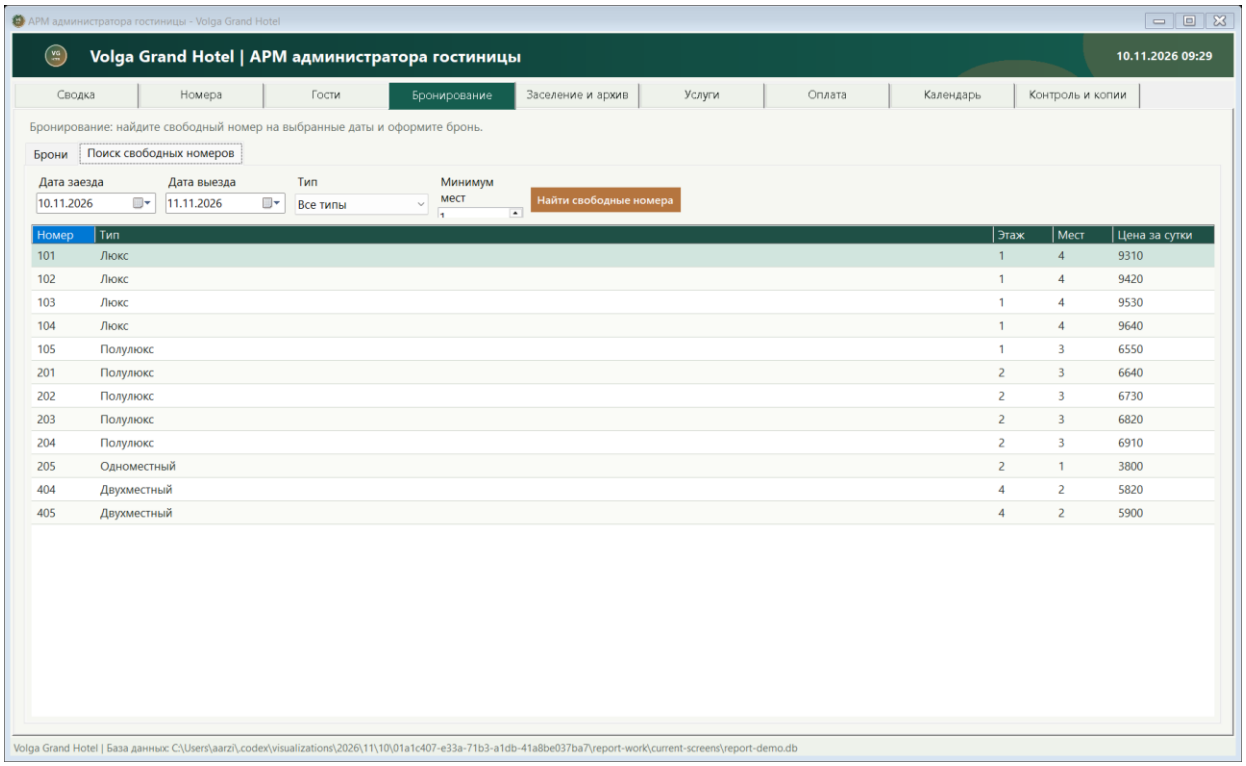


Рисунок Г.4 — Поиск свободных номеров

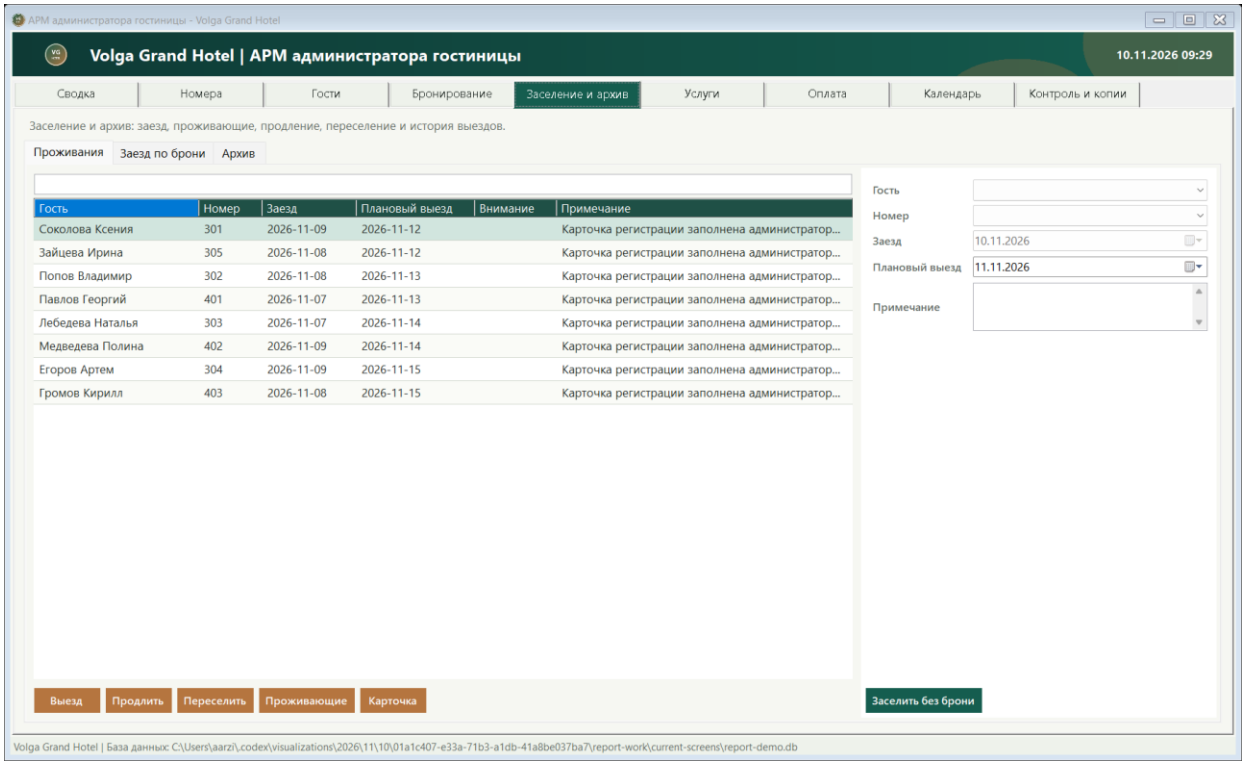


Рисунок Г.5 — Проживания и управление заездом

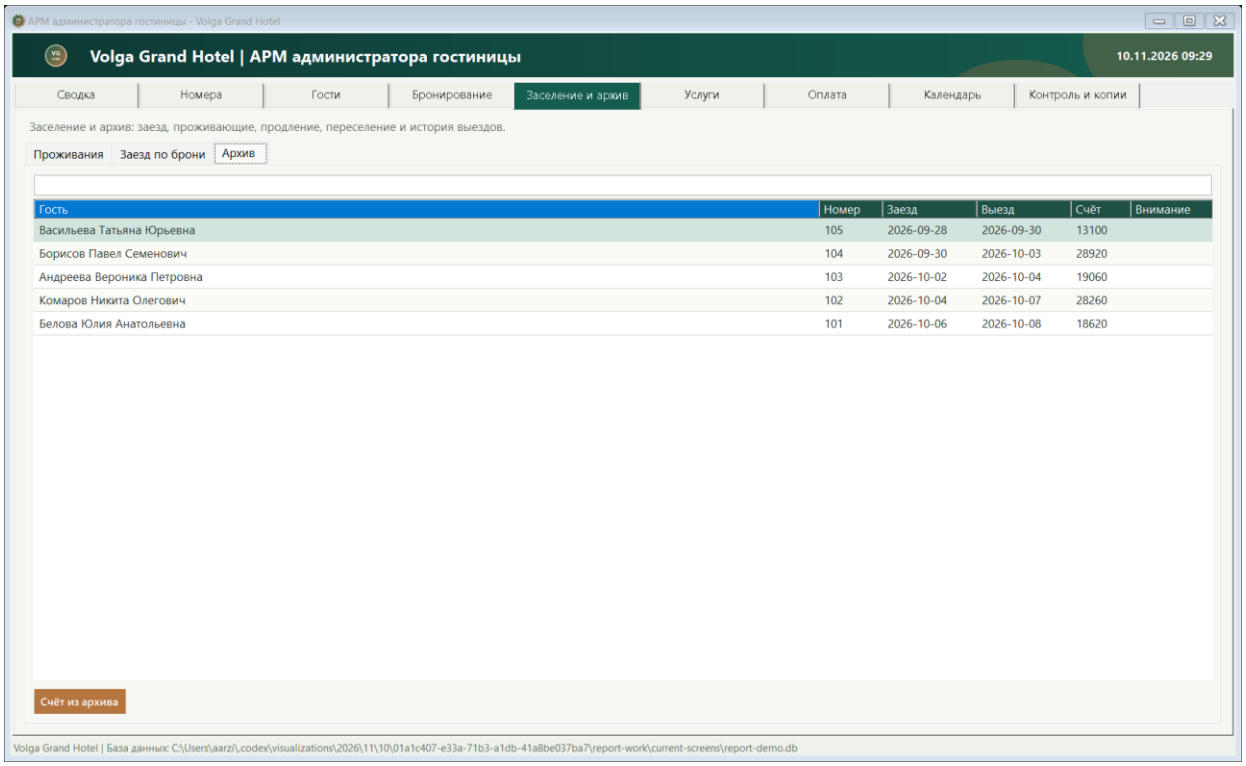


Рисунок Г.6 — Архив выездов

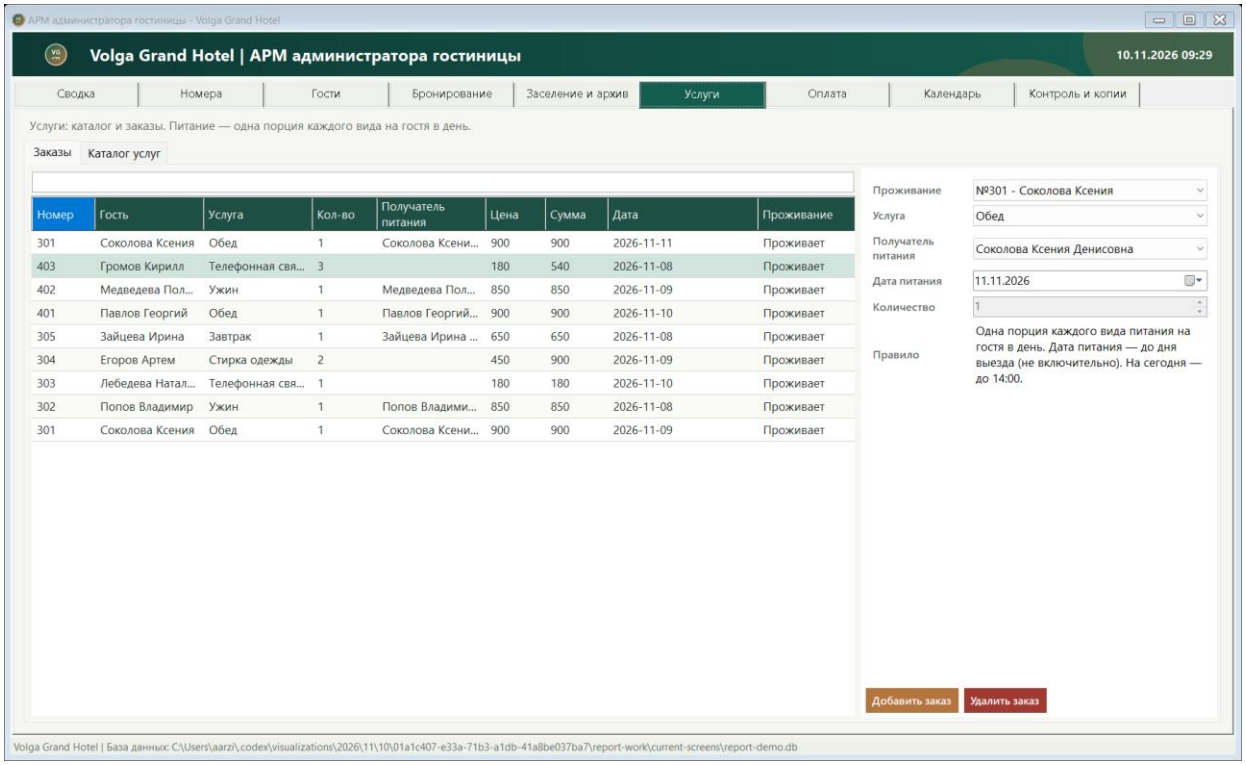


Рисунок Г.7 — Выбор дня и получателя питания

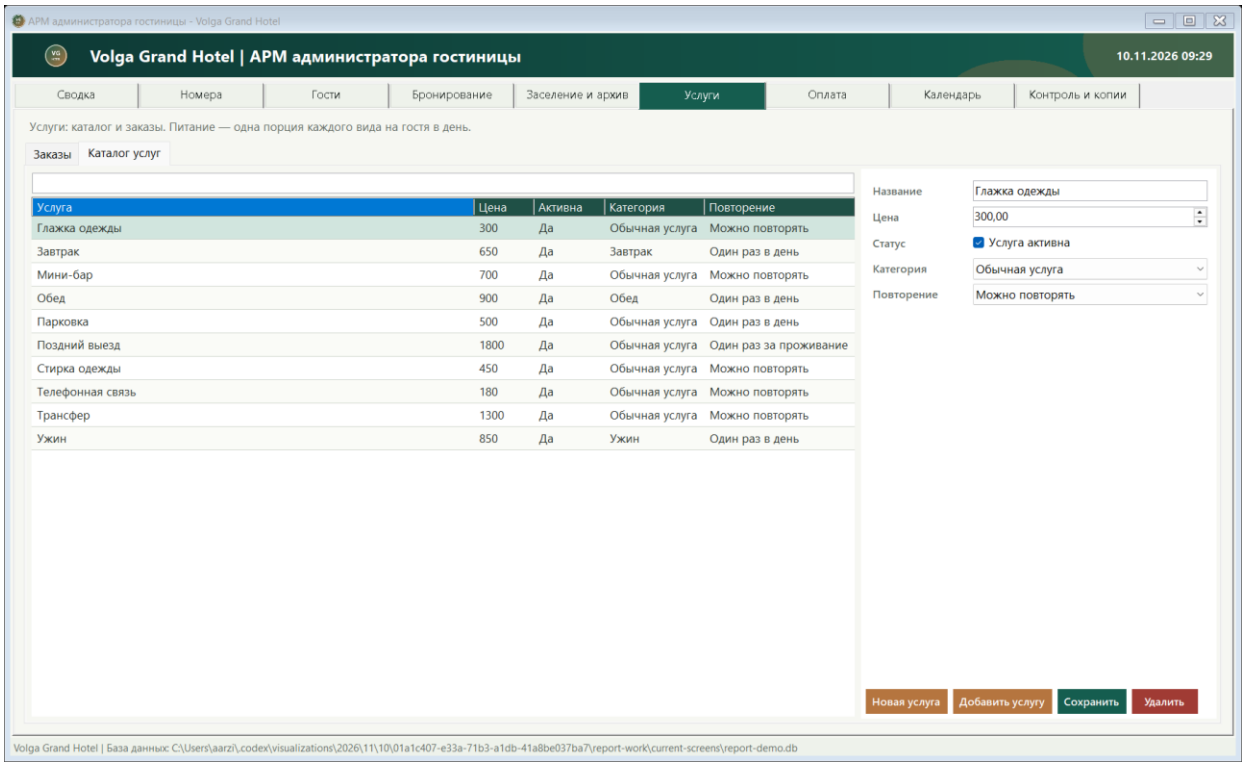


Рисунок Г.8 — Каталог услуг и правила повторения

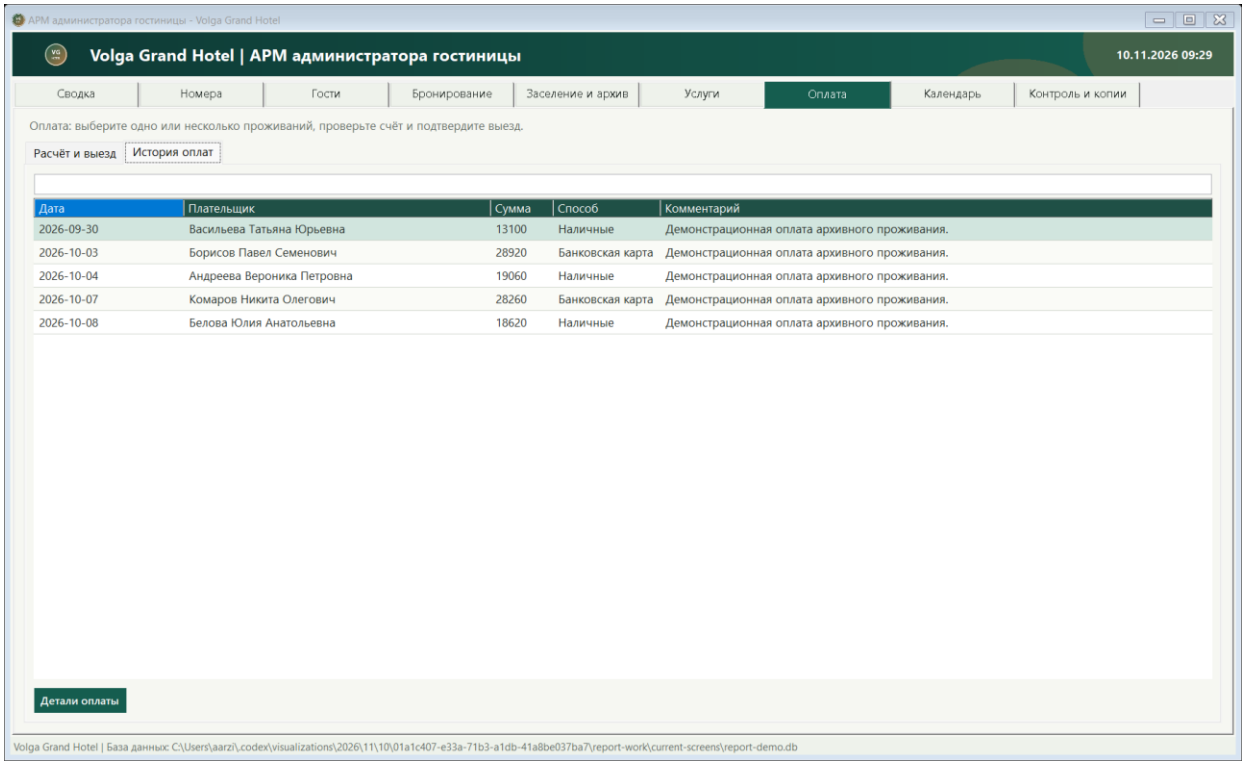


Рисунок Г.9 — История оплат

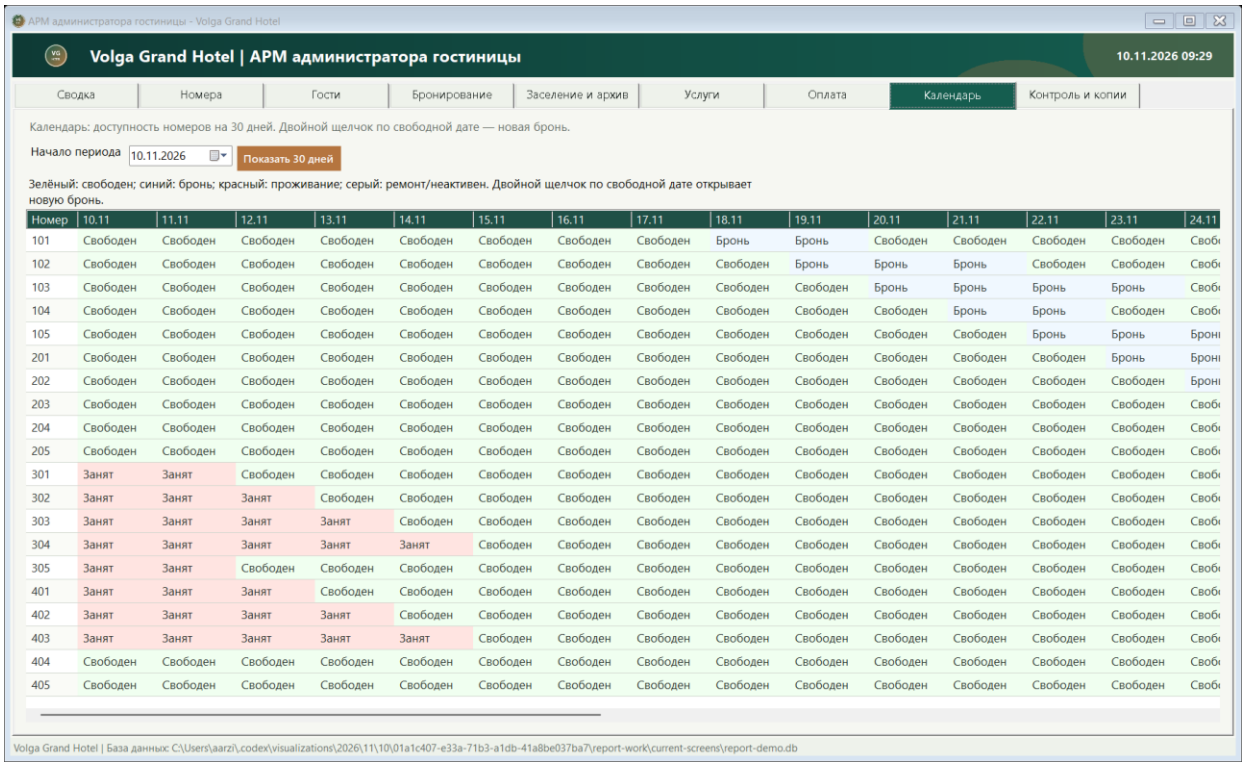


Рисунок Г.10 — Календарь доступности

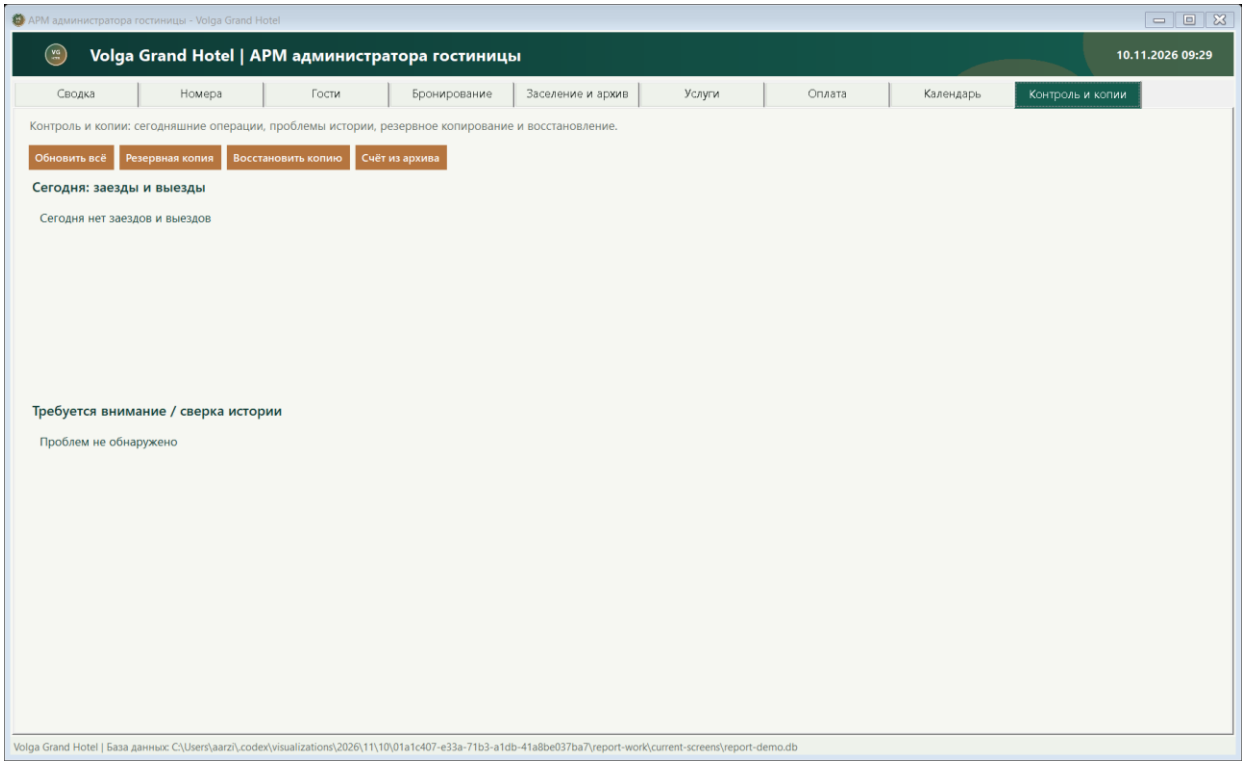


Рисунок Г.11 — Контроль и резервные копии

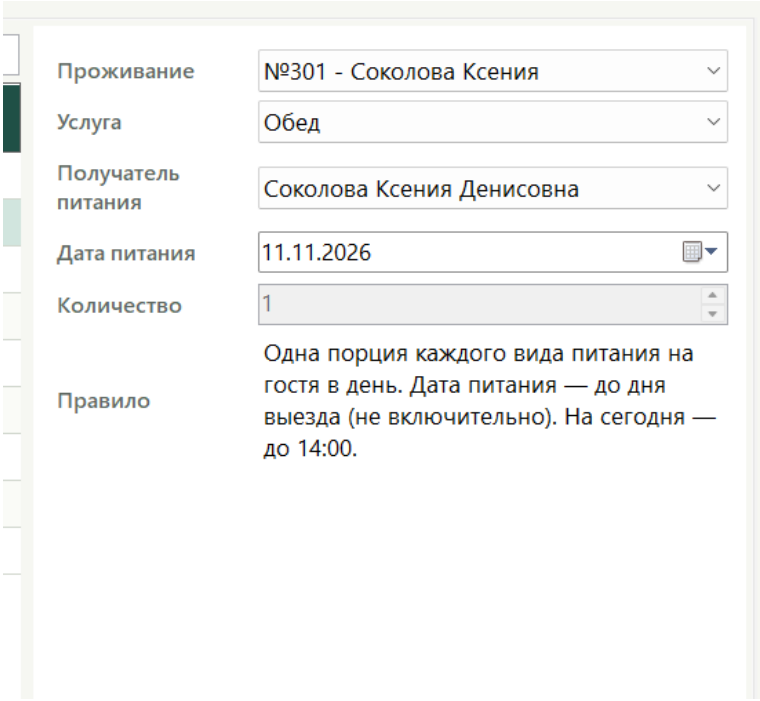


Рисунок Г.12 — Поля даты питания и ограничения времени